AI-Powered Waste Sorting & Recycling Assistant

Domain: sustainability and waste management

Subtopic: photo recognition with regional sorting guidance, an AI coach and submission analytics

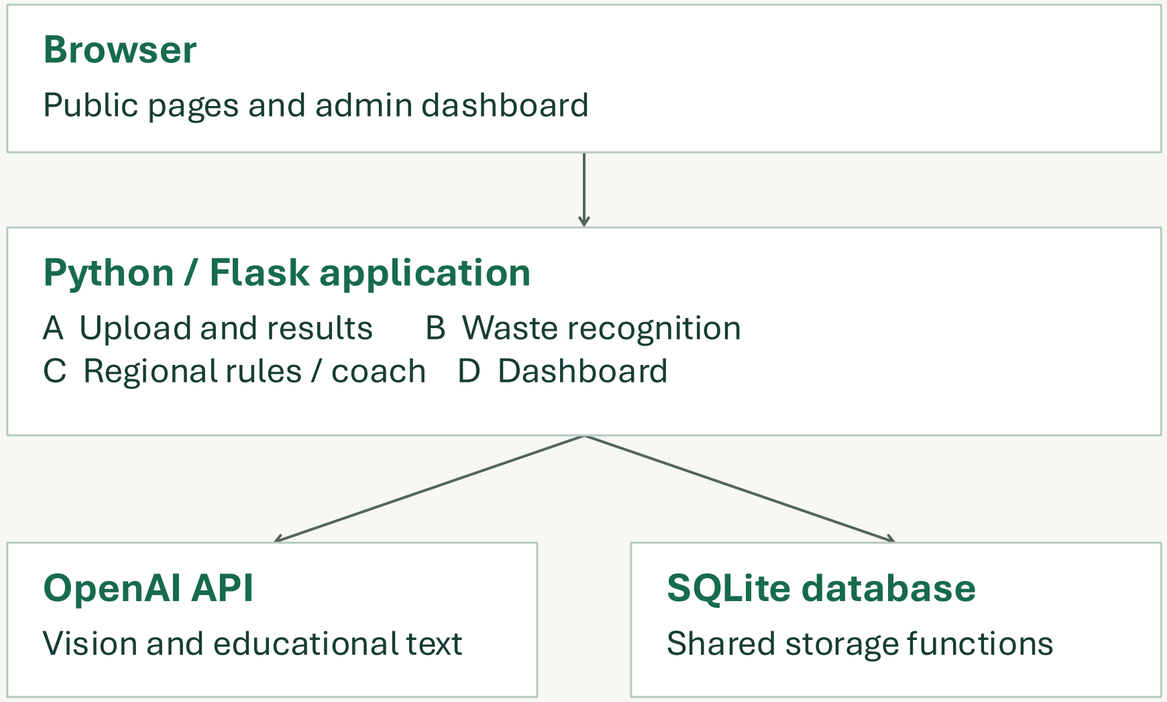
Team and subsystem distribution

Presenter	Subsystem
Adel	A Public web interface
Mahdi	B Waste recognition
Jayesh	C Regional rules and AI coach
Rashi	D Municipal dashboard

Each member presents one subsystem:
10 functional + 10 non-functional requirements,
3 domain classes, storage and a wireframe.

All four share development, reviews and testing.
Workload balance is a proposal to confirm together.

First architecture draft



JSON between modules and API. Photos stay in memory.
Docker runs the app, with a volume for the database.

A Public website

ADEL ABASALAM ESMAIL BASALAMA

Receives a photo and region, coordinates sorting, and displays the result.

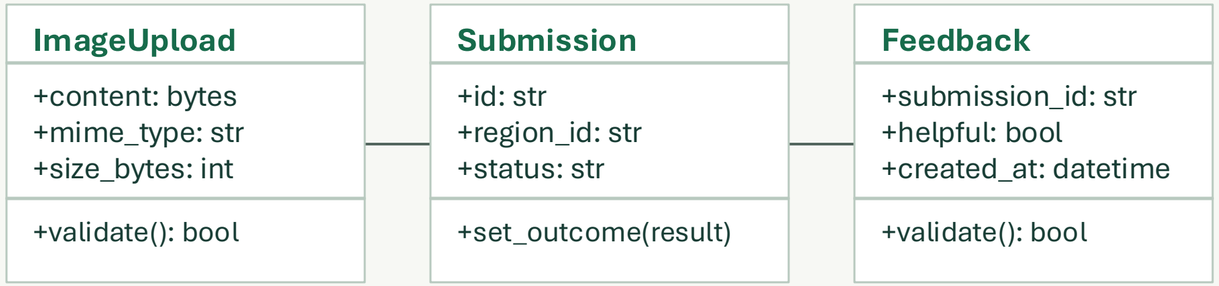
Functional requirements

A-F01	Accept waste-item image uploads.
A-F02	Support JPG, JPEG, and PNG.
A-F03	Show image before submission.
A-F04	Let users choose their city/region.
A-F05	Reject invalid/unsupported images.
A-F06	Forward images to recognition service.
A-F07	Display item and waste category.
A-F08	Show region-specific disposal advice.
A-F09	Explain uncertainty and allow retry.
A-F10	Start another identification.

Non-functional requirements

A-N01	Load main page within 3 seconds
A-N02	Support 50 concurrent users
A-N03	Fit phone, tablet, and desktop
A-N04	Support recent major browsers
A-N05	Use HTTPS for all connections
A-N06	Reject image files over 10MB
A-N07	Use clear English and error messages
A-N08	Target 99% scheduled availability
A-N09	Meet WCAG 2.1 AA accessibility
A-N10	Repeated clicks create only one active request.

Domain class diagram: 3 classes



Submission 1 has ImageUpload 1 and Feedback 0..1

Validated upload calls B, then C. A stores the outcome and displays the result.

Data management

Writes Submission and Feedback.
Reads Region and the final decision.
Photo bytes stay in memory.
UUID / enum: TEXT, bool: INTEGER.
Times: UTC ISO strings.

Interface prototype

Region: [Choose A Region v]

Photo: [Choose image]

[Recognize item]

Result / guidance / feedback

B Waste recognition

Mahdi Yahia

Identifies the item and material, standardizes labels, and flags uncertainty.

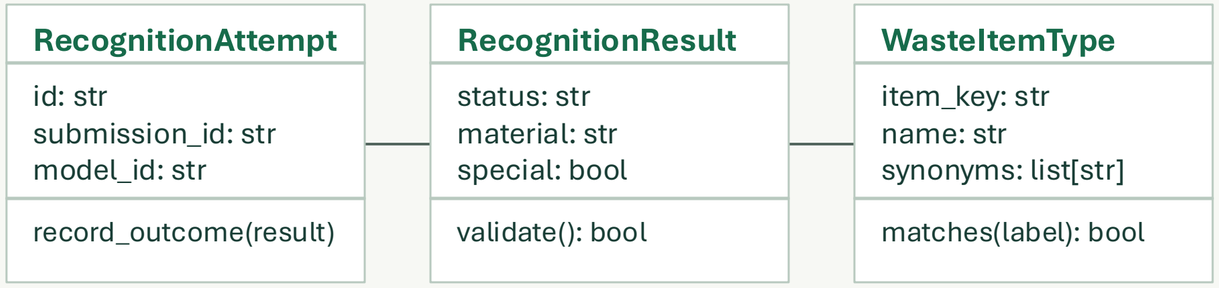
10 functional requirements

- **B-F01** Receive the validated image and submission ID.
- **B-F02** Map item synonyms to a standard item key.
- **B-F03** Resize the image and remove metadata.
- **B-F04** Identify a supported waste item using AI.
- **B-F05** Assign a material label from the taxonomy.
- **B-F06** Report clean, soiled or unknown condition.
- **B-F07** Return uncertain / unsupported when needed.
- **B-F08** Flag suspected batteries and electronics.
- **B-F09** Return the agreed structured recognition result.
- **B-F10** Handle AI timeout, refusal and invalid output.

10 non-functional requirements

- **B-N01** Item accuracy ≥ 80% on 50 unseen photos.
- **B-N02** Flag ≥ 9 of 10 ambiguous test photos.
- **B-N03** p95 ≤ 12 s over 30 calls, hard timeout 15 s.
- **B-N04** 100% of accepted results pass the schema.
- **B-N05** Store no photo in application files or logs.
- **B-N06** API keys stay out of browser, Git and logs.
- **B-N07** ≤ 1 vision call per submission, no auto-retry.
- **B-N08** Failure fixtures never crash or block service.
- **B-N09** Separate module, configurable model / timeout.
- **B-N10** ≥ 10 unit tests using a mocked AI service.

Domain class diagram: 3 classes



Data management

Writes RecognitionAttempt metadata.
Result fields share the attempt row.
Reads WasteItemType catalogue.
Image / raw AI payload: temporary.
Item key: TEXT, special: INTEGER.

Interface prototype

Recognition result (demo)

Item: beverage_can

Material: metal Condition: clean

Status: recognized Special: no

[Continue to local guidance]

B sends a processed image to vision, validates the response, then passes item facts to C.

C Regional rules and education

Member 3

Resolves regional disposal rules and adds an AI sustainability explanation.

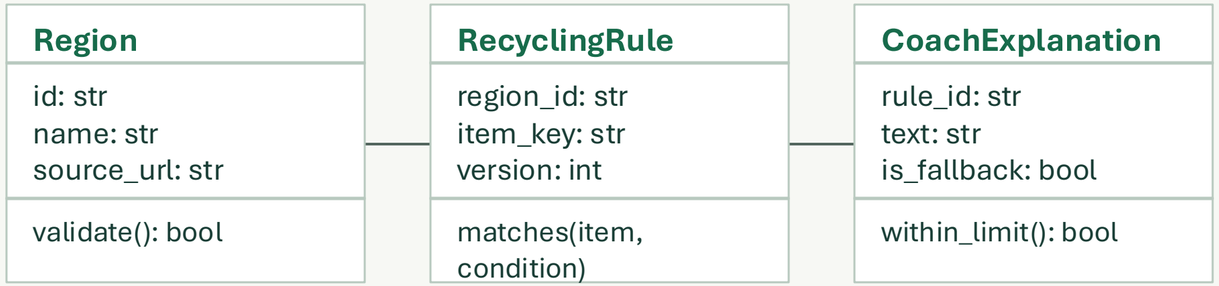
10 functional requirements

- **C-F01** Store regions and official source links.
- **C-F02** Create / edit rules by region, item and condition.
- **C-F03** Match recognized items to local rules.
- **C-F04** Return category, bin label and preparation steps.
- **C-F05** Handle special items and missing rules explicitly.
- **C-F06** Include rule source, review date and version.
- **C-F07** Generate a plain-language AI explanation.
- **C-F08** Keep the rule decision independent of AI text.
- **C-F09** Use approved text when education fails.
- **C-F10** Preserve rule versions used in past decisions.

10 non-functional requirements

- **C-N01** All 20 regional-rule fixtures match expectations.
- **C-N02** Lookup p95 < 200 ms with 1,000 rules.
- **C-N03** Foreign keys reject invalid region references.
- **C-N04** One active rule per region / item / condition.
- **C-N05** Every active rule has complete provenance.
- **C-N06** All 20 reviewed explanations respect the rule.
- **C-N07** Coach text ≤ 100 words or use fallback.
- **C-N08** Education timeout 8 s, fallback within 1 s.
- **C-N09** Every rule change requires authorized access.
- **C-N10** New regions and rules require no code change.

Domain class diagram: 3 classes



Data management

Writes Region and RuleRevision.
Saves CoachExplanation with rule ID.
Old rule versions remain immutable.
Category: nullable TEXT enum.
Sources: URLs, review dates: TEXT.

Interface prototype

Rule editor (admin, demo)

Region: Demo A Item: [select]

Condition: [select] Bin: [text]

Source URL: [ ]

[Save version] [Preview tip]

C looks up the rule, determines the bin, and asks the coach to explain that approved rule.

D Municipal dashboard

Rashi Jain

Provides authorized analysis of submitted items, outcomes and feedback.

10 functional requirements

D-F01	Show recyclable share for a selected period.
D-F02	Show waste composition by category as a chart.
D-F03	Filter data by region.
D-F04	Count the three ordinary disposal categories.
D-F05	Filter by period: day, week, month or year.
D-F06	Count confused items (< 60% or user-corrected).
D-F07	Show the 10 most frequently scanned items.
D-F08	Show recyclable share over time (line chart)
D-F09	Export filtered aggregates as CSV.
D-F10	Show waste composition over time.

10 non-functional requirements

D-N01	Pages load < 3 s with 100,000 submissions.
D-N02	Read-only: never changes or deletes data.
D-N03	Reports expose aggregate data only.
D-N04	Missing data shown as “Unknown”, no crash.
D-N05	New user finds recyclable share in < 1 min.
D-N06	Supports 10+ regions without code changes.
D-N07	Category percentages add up to 100%.
D-N08	Usable on desktop and tablet (≥ 768 px).
D-N09	Separate modules: data, calculations, charts.
D-N10	Colour-blind-safe charts with value labels.

Tech Stack

- python,
- pandas,
- matplotlib(for charts),
- stramlit (interface)/react
- vscode,
- git

Domain class diagram: 4 classes

Submission

```
submission_id: str,
region_id: str,
created_at: datetime,
category: str,
confidence: float
```

StatisticsCalculator

- recyclable_waste() - trend over time : float
- waste_composition() the pie chart : dict
- top_scanned_items() the bar chart : dataframe
- confused_items() : dataframe
- trend_over_time() the line chart : dataframe

Data management

Reads Submission and Feedback.
Uses shared outcome-save function.
Filters / aggregates stay in memory.
Percentages derive from SQL counts.

Interface prototype

Dashboard (demo counts)

Region: Demo A Dates: [range]

Recyclable share: 24/40 = 60%

Top uncertain 5

[Daily trend] [Export CSV]

D authenticates, applies filters and calculates counts. This describes submissions, not city recycling.